

P2649R0

2022-10 Library Evolution Poll Outcomes

Published Proposal, 2022-11-11

Authors:

[Bryce Adelstein Lelbach \(he/him/his\) — Library Evolution Chair](#) (NVIDIA)

[Fabio Fracassi — Library Evolution Vice Chair](#) (CODE University of Applied Sciences)

[Ben Craig — Library Evolution Vice Chair](#) (NI)

[Inbal Levi — Ranges Chair \(SG9\) and Library Mailing List Review Manager](#)

Source:

[GitHub](#)

Issue Tracking:

[GitHub](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

WG21

Table of Contents

1 Introduction

2 Poll Outcomes

3 Selected Poll Comments

3.1 C++26 and Technical Specification Polls

3.1.1 [P2631R0] Publish The Library Fundamentals v3 Technical Specification Now

3.1.2 Don't Publish The Library Fundamentals v3 Technical Specification

3.1.3 [P2511R2] Beyond operator(): NTTP Callables In Type-Erased Call Wrappers

References

Informative References

§ 1. Introduction

In 2022-10, the C++ Library Evolution group conducted a series of electronic decision polls [P2648R0]. This paper provides the results of those polls and summarizes the results.

In total, 24 people participated in the polls. Some participants opted to not vote on some polls. Thank you to everyone who participated, and to the proposal authors for all their hard work!

§ 2. Poll Outcomes

- SF: Strongly Favor.
- WF: Weakly Favor.
- N: Neutral.
- WA: Weakly Against.
- SA: Strongly Against.

Poll	SF	WF	N	WA	SA	Outcome
Poll 1.1: Return [P2539R3] print To Terminal Synchronization to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	8	11	0	1	0	Consensus in favor.
Poll 1.2: Send the proposed resolution to [LWG3515] [stacktrace.basic.nonmem]: operator<< Should Be Less Templated to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	9	5	0	1	1	Consensus in favor.
Poll 2.1: Publish the Library Fundamentals v3 Technical Specification now as per [P2631R0].	6	5	0	3	2	Weak consensus in favor. We will proceed with publishing the Library Fundamentals v3 Technical Specification now as per [P2631R0].

Poll 2.2: Don't publish the Library Fundamentals v3 Technical Specification, but evaluate its contents for inclusion into a future International Standard.	4	4	1	3	5	No consensus. We will proceed with publishing the Library Fundamentals v3 Technical Specification now as per [P2631R0].
Poll 2.3: Return [P1083R6] resource_adaptor to Library Working Group for C++26, classified as an addition ([P0592R4] bucket 3 item).	1	7	1	0	1	Consensus in favor.
Poll 2.4: Send [P2587R3] to_string Or Not to_string to Library Working Group for C++26, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	11	6	0	0	0	Unanimous consensus in favor.
Poll 2.5: Send [P2495R1] Interfacing stringstream With string_view to Library Working Group for C++26, classified as an addition ([P0592R4] bucket 3 item).	6	8	2	2	0	Consensus in favor.
Poll 2.6: Send [P2510R3] Formatting Pointers to Library Working Group for C++26, classified as an addition ([P0592R4] bucket 3 item).	6	9	1	0	1	Consensus in favor.
Poll 2.7: Send [P2572R0] std::format Fill Character Allowances to Library Working Group for C++26, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	2	10	1	0	0	Strong consensus in favor.
Poll 2.8: Send [P2511R2] Beyond operator(): NTTP Callables In Type-Erased Call Wrappers to Library Working Group for C++26, classified as an addition ([P0592R4] bucket 3 item).	4	7	3	1	2	No consensus.
Poll 2.9: Send [P2592R2] Hashing Support For chrono Value Classes to Library Working Group for C++26, classified as an	8	8	1	1	0	Consensus in favor.

improvement of an existing feature ([P0592R4] bucket 2 item).						
Poll 2.10: Send [P0543R2] Saturation Arithmetic to Library Working Group for C++26, classified as an addition ([P0592R4] bucket 3 item).	4	11	0	1	0	Consensus in favor.
Poll 2.11: Send [P0870R4] is_convertible_without_narrowing to Library Working Group for C++26, classified as an addition ([P0592R4] bucket 3 item).	9	6	1	0	0	Strong consensus in favor.
Poll 2.12: Send [P2614R1] Deprecate numeric_limits::has_denorm to Library Working Group for C++26, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	6	7	2	0	0	Poll withdrawn due to incorrect ship vehicle. It will be retaken targeting C++23 in the next electronic polling period.

§ 3. Selected Poll Comments

§ 3.1. C++26 and Technical Specification Polls

§ 3.1.1. [P2631R0] Publish The Library Fundamentals v3 Technical Specification Now

The decision to publish a TS (or not) rests with the plenary, not LEWG. To the extent this poll is meaningful, I prefer to publish than to leave it in limbo.

— Strongly Favor

We should publish this material as-is now instead of rebasing it again. After the publication of LibFunTSv3 we should have a plenary vote on whether we want to continue doing LibFunTS in the future.

— Strongly Favor

This vote is aligned with my personal view, but I am choosing that vote in a way that would lead to plenary (full committee) decision on the treatment of LFTSv3. The start and additions to the document was done by plenary votes.

— Strongly Favor

We committed to this TS; we put work into it; paper authors rightly expect their work to get published. There's no apparent reason to delay the (already late) TS any further.

— Strongly Favor

It does not serve the broader community to take the document already prepared and keep it as an internal paper indefinitely.

— Weakly Favor

The work is done, there are benefits to having a TS, so let's just go ahead and push the button.

— Weakly Favor

This option seems like the least work for the most gain. It's a bit troubling that the document has been around for so long that the way WG21 treats TSeS has changed. On the other hand, if somebody is willing to do the work of rebasing and updating the document, then the document must have value for someone.

— Weakly Favor

I don't think my experience is enough to estimate the implications

— Did Not Participate

We should evaluate the parts of the specification for inclusion into the standard and abandon the TS.

— Weakly Against

The TS model does not work very well for libraries, we should get the useful facilities into the standard and stop working on the rest

— Weakly Against

LFTSes fail to provide more feedbacks than regularly updated papers, and are generally a waste of time. Indecisions in regards to specific papers cannot be solved by more paperwork. Bring back individual papers for consideration for C++26.

— Strongly Against

1. LFTS is not Fit-For-Purpose.
2. It is unclear why we are polling this, given that (a) it utterly failed to reach consensus in LEWG (3-5-3-4-2) & (b) is likely to be straw polled in Kona plenary regardless of the outcome of this electronic poll.

My objection is that the intent of having that LFTS was to answer open questions before deciding whether or not to put these libraries into the IS. With the exception of `scope_guard`, every library in the LFTS has had at least five years to get feedback; some have had seven years. AFAIK, the purpose of the LFTS has not changed, so why are we not deciding what to do with the libraries in it?

If the purpose has changed and now WG21 wants to keep these libraries in the LFTS forever, then we should state that clearly and unambiguously and ultimately vote on that, and stop pretending that the LFTS has a different purpose.

Based on this history, I don't believe that LFTS3 will get us any feedback on the one new feature proposed for it (`scope_guard`) either.

The LFTS3 is itself misleading. [general.plans] states:

"The C++ committee intends to release new versions of this technical specification periodically, containing the library extensions we hope to add to a near-future version of the C++ Standard. Future versions will define their contents in `std::experimental::fundamentals_v4`, `std::experimental::fundamentals_v5`, etc., with the most recent implemented version inlined into `std::experimental`."

While some committee members are okay with publishing knowingly misleading statements in non-normative sections of documents, I am not and will vote strongly against approving such a document.

Another misleading statement from [general.plans]:

"When an extension defined in this or a future version of this technical specification represents enough existing practice, it will be moved into the next version of the C++ Standard by removing the `experimental::fundamentals_vN` segment of its namespace and by removing the `experimental/` prefix from its header's path."

If that isn't our plan, I cannot sign off on it.

— Strongly Against

§ 3.1.2. Don't Publish The Library Fundamentals v3 Technical Specification

LFTSes fail to provide more feedbacks than regularly updated papers, and are generally a waste of time. Indecisions in regards to specific papers cannot be solved by more paperwork. Bring back individual papers for consideration for C++26.

— Strongly Favor

The TS model does not work very well for libraries, we should get the useful facilities into the standard and stop working on the rest

— Strongly Favor

Given that we have had 5-7 years to get feedback on all of the libraries in this document except for `scope_guard`, the time has come to eliminate it and decide once and for all if the features in this document should be added to the IS or not.

As for `scope_guard`, given the lack of feedback we've gotten on the other features over the last 5-7 years, a LFTS3 with only `scope_guard` does not seem to be a good use of our resources. LEWG should come up with a better plan to make the hard design decisions necessary to add `scope_guard` to the IS.

— Weakly Favor

If we don't publish the TS, we should actively consider the contents for inclusion in the IS rather than just abandoning them.

— Weakly Favor

I don't think my experience is enough to estimate the implications, but this seems like the cautious route

— Weakly Favor

We should evaluate the parts of the specification for inclusion into the standard and abandon the TS.

— Weakly Favor

I wouldn't mind if we did this, as long as this includes the possibility of breaking up the document into separate proposals, or dropping the whole thing.

— Neutral

I have not adequately followed discussion of this paper and do not have a sufficiently informed opinion on it.

— Did Not Participate

I have not participated in the corresponding discussions. But, we should absolutely consider its content for inclusion in a future standard!

— Did Not Participate

Presumably this is meant to lead to WG21 in plenary moving to withdraw the work item (since otherwise it would just be a poll for the status quo). There is an argument that failure of Poll 1 would motivate such a move, but that hasn't happened yet.

— Weakly Against

The choices where we don't publish the TS don't help anyone.

— Weakly Against

We shouldn't throw the baby out with the bathwater, so I'm SA not publishing LibFunTSv3. But yes, we should evaluate which features we actually want in the IS (the rest should be removed from the TS). IMNSHO there should be a plenary poll whether we want to commit to doing LibFunTSv4.

— Strongly Against

This vote is aligned with my personal view, but I am choosing that vote in a way that would lead to plenary (full committee) decision on the treatment of LFTSv3. The start and additions to the document were done by plenary votes.

— Strongly Against

The decision to publish a TS (or not) rests with the plenary, not LEWG. To the extent this poll is meaningful, I prefer to publish than to leave it in limbo.

— Strongly Against

The latter is not dependent on the former. I.e. evaluate LFTS contents for IS inclusion as soon as it makes sense. But that's an orthogonal issue from getting the LFTS out.

— Strongly Against

§ 3.1.3. [\[P2511R2\]](#) Beyond operator(): NTTP Callables In Type-Erased Call Wrappers

Broadens usability of type-erased function wrappers, making it possible to wrap more "functions".

— Strongly Favor

Easier, safer and more efficient

— Strongly Favor

I've wanted to do this in my own code and it'd be good to get away from forcing choosing between member functions which are descriptive and require boilerplate to use with functional interfaces and operator() which works with functional interfaces but which isn't descriptive.

— Weakly Favor

This looks like a useful facility with parallels in other programming languages. The paper explains why an expansion of lambdas could not straightforwardly replace this.

— Weakly Favor

I believe I understand how this is being used and I can imagine it improving the compiler's ability to optimize code.

— Weakly Favor

The generalization of `std::integer_constant` is needful, but making it simultaneously generic of name and specifically recognized by the `std::function`-like components is a bit surprising.

— Neutral

I'm not sold on this.

The desire expressed by some for terser lambdas is not something we should try to paper over in the library. This fails in the presence of overload sets and the name chosen aren't great either. I wonder how often this would end up being used, writing a lambda is probably faster, if only because that's the intuitive approach powered by muscle memory. If anything the examples suffer more of the clunkyness of `std::forward`. Given <https://wg21.link/p0644r1>:

```
pack.start([&obj] (T &&... args) { return obj.send(>>args...); });
```

— Neutral

This feels like a significant complication to the API, without any attempt to quantify the benefit.

— Neutral

I have not adequately followed discussion of this paper and do not have a sufficiently informed opinion on it.

— Did Not Participate

Though I support the intent, I am not convinced that the `non_type<>` wrapper is the right thing here.

— Weakly Against

Seems very weakly motivated. If we had `constexpr` function parameters, would be pointless. Doesn't seem worthwhile to pursue.

— Strongly Against

In contrast to `function_ref`, I do not see this constructor as necessary, as `bind_front` with empty functor could be applied to achieve the same effects, i.e. it would be sufficient to make `nttp` callable. This would also benefit other use cases, that want to bind a member with an object using `bind_front`.

— Strongly Against

§ References

§ Informative References

[P0543R2]

Jens Maurer. *Saturation arithmetic*. 18 September 2022. URL: <https://wg21.link/p0543r2>

[P0592R4]

Ville Voutilainen. *To boldly suggest an overall plan for C++23*. 25 November 2019. URL: <https://wg21.link/p0592r4>

[P0870R4]

Giuseppe D'Angelo. *A proposal for a type trait to detect narrowing conversions*. 23 September 2020. URL: <https://wg21.link/p0870r4>

[P1083R6]

Pablo Halpern. *Move resource_adaptor from Library TS to the C++ WP*. 9 July 2022. URL: <https://wg21.link/p1083r6>

[P2495R1]

Michael Hava. *Interfacing stringstreams with string_view*. 14 September 2022. URL: <https://wg21.link/p2495r1>

[P2510R3]

Mark de Wever. *Formatting pointers*. 23 May 2022. URL: <https://wg21.link/p2510r3>

[P2511R2]

Zhihao Yuan. *Beyond operator(): NTPP callables in type-erased call wrappers*. 15 August 2022. URL: <https://wg21.link/p2511r2>

[P2572R0]

Tom Honermann. *std::format() fill character allowances*. 11 June 2022. URL: <https://wg21.link/p2572r0>

[P2587R3]

Victor Zverovich. *to_string or not to_string*. 28 August 2022. URL: <https://wg21.link/p2587r3>

[P2631R0]

Alisdair Meredith, Bryce Adelstein Lelbach, Jonathan Wakely. *[Publish TS Library Fundamentals v3 Now!](#)*. 12 September 2022. URL: <https://wg21.link/p2631r0>